

Movie Recommendation System Report

Introduction

Recommendation systems play a vital role in shaping user experience on content-rich platforms such as Netflix, Amazon Prime, and YouTube. By analyzing user preferences and behaviors, these systems reduce information overload and increase user engagement.

In this project, we implement and compare three major types of recommendation algorithms using the MovieLens 100K dataset:

1. User-Based Collaborative Filtering: Identifies users with similar rating behaviors and recommends items they've liked.
2. Item-Based Collaborative Filtering: Recommends items similar to what the user has already rated positively.
3. Pixie-Inspired Random Walk Algorithm: A graph-based algorithm that explores deeper relationships via random walks on a bipartite graph.

These approaches demonstrate a variety of strategies in personalization, from similarity-based heuristics to structure-aware graph exploration.

Dataset Description

We use the MovieLens 100K dataset, a benchmark dataset widely used in recommendation system research.

Dataset Components

- u.data: Contains 100,000 ratings (user_id, movie_id, rating, timestamp).
- u.item: Metadata for 1,682 movies, including titles, genres, and release dates.
- u.user: Information on 943 users including age, gender, occupation, and zip code.

Key Statistics

- Users: 943
- Movies: 1,682
- Ratings: 100,000
- Sparsity: Only ~6.3% of all possible ratings are filled.

Preprocessing Steps

- Parsed tab-delimited files using pandas.
- Merged data to form a user-item matrix.
- Converted raw data into:
 - User-by-movie matrix for collaborative filtering.
 - Bipartite graph (user → movie and vice versa) for the Pixie algorithm.

Methodology

1. User-Based Collaborative Filtering

Users who rate items similarly in the past are likely to agree in the future.

Cosine similarity between user rating vectors.

```
user_similarity = cosine_similarity(user_movie_matrix.fillna(0))
```

```
user_sim_df = pd.DataFrame(user_similarity, index=user_movie_matrix.index,  
columns=user_movie_matrix.index)
```

Recommendation Strategy:

- Select top-K similar users.
- Aggregate their ratings for unseen items.
- Recommend top-rated unseen items.

2. Item-Based Collaborative Filtering

If you liked movie A, you'll probably like movie B if many users rated both similarly.

Cosine similarity between columns (i.e., movies) in the user-item matrix.

```
item_similarity = cosine_similarity(user_movie_matrix.T.fillna(0))
```

```
item_sim_df = pd.DataFrame(item_similarity, index=user_movie_matrix.columns,  
columns=user_movie_matrix.columns)
```

3. Pixie-Inspired Graph-Based Random Walk

Represent users and items as nodes in a bipartite graph. Use weighted random walks to surface connected nodes (items) from a starting user.

Why Graph-Based?

- Explores deeper transitive relationships (user → movie → user → movie...).
- Captures long-range signals missed by direct similarity.

Graph Representation:

- Adjacency list stored as a dictionary.
- Users and movies form two sets of nodes, with edges indicating interactions (ratings).

Random Walk Strategy:

- Initialize walker at a user node.
- Walk alternately between movies and users.
- Each visit to a movie increases its score.
- After N walks, rank movies by frequency.

Implementation Details

Tools and Libraries:

- Pandas: Data cleaning and merging.
- NumPy: Numerical computations.
- Scikit-learn: Cosine similarity.
- Defaultdict (from collections): For adjacency lists and counters.

Modular Structure:

- Each algorithm encapsulated in functions.
- Input flexibility to allow dynamic recommendations.
- Minimal dependencies (no NetworkX or external graph libraries).

Reusability:

- Graph and similarity matrices can be cached for efficiency.
- Ready for integration with a front-end or API.

Results and Evaluation

Example Recommendations:

- **User-Based (for User 10):**
 - *Star Wars*
 - *Pulp Fiction*
 - *Raiders of the Lost Ark*
- **Item-Based (related to *Toy Story*):**
 - *Aladdin*
 - *Lion King*
 - *Beauty and the Beast*
- **Pixie Algorithm:**
 - Generated diverse and sometimes serendipitous movie suggestions by exploring user-movie-user chains.

Comparative Insights:			
Metric	User-Based	Item-Based	Pixie (Random Walk)
Diversity	Moderate	Low	High
Cold Start Sensitivity	High	High	Lower
Computation Time	Medium	Medium	High (if many walks)
Interpretability	High	High	Moderate

Limitations

- No explicit precision/recall/F1 evaluation in this version.
- Random walks may yield unstable results with small walk counts.
- Cold-start problem still present in CF approaches.
- Execution speed could degrade for larger graphs without optimization.

Conclusion

This project successfully demonstrates the practical implementation of multiple recommendation techniques on a real-world dataset. Each method has unique advantages and trade-offs.

Key Takeaways:

- User- and item-based methods are effective but limited by direct similarity.
- Pixie Offers enhanced diversity and recommendation depth using structural insights.
- Graph-based models offer deeper, more exploratory recommendations.
- Each method has trade-offs in performance and accuracy.

Potential Improvements:

- Implement hybrid models combining collaborative filtering with Pixie.
- Add content-based filtering using movie genres, cast, etc.
- Use implicit feedback (e.g., views, browsing history).

Real-World Applications

Streaming Services (e.g., Netflix, Disney+)

- Use hybrid models combining collaborative filtering (CF), content-based filtering, and deep learning. CF suggests shows based on similar users' viewing habits. Graph-based approaches help uncover lesser-known but highly relevant content.

E-Commerce (e.g., Amazon, Etsy)

- Item-based CF recommends products similar to those in the cart or past purchases. Pixie-like algorithms surface related products based on shopping graph traversal.

Social Media (e.g., YouTube, TikTok)

- Algorithms walk user-content graphs to suggest engaging content. Emphasize exploration and engagement through serendipitous recommendations.

News Aggregators (e.g., Google News, Flipboard)

- Build graphs of users, topics, and articles. Recommend stories using collaborative signals and temporal patterns.